

# Discussion on Deployment

17-316/616 Fall 2025

AI Tools for Software Development

<https://ai-developer-tools.github.io>

Austin Henley and Andrew Begel

# P6 Questions

- What questions do you have about P6: Deployment?

# Reflection prompts this week

- Could an LLM be integrated into your continuous integration pipeline to automatically review code, generate tests, or update tests whenever new code is committed? How would you do it? What are the potential benefits and risks to your production environment?
- Cloud providers like Amazon, Microsoft, and Google deliberately design their systems to promote vendor lock-in. This makes it very difficult to switch from one cloud to another. Speculate how you could use an LLM to support dynamically switching your app between cloud providers without bothering any human software developers. What would you need to do?
- Deployment to the cloud is much more complex than deploying on a single server in your garage. Ask your LLM to explain to you why this complexity is justified. After reading its explanation, engage your human brain to discuss two pros and two cons to cloud deployment vs. deployment on your own server.
- Imagine your company uses an LLM-driven CI/CD system that can automatically decide when to deploy, roll back, or patch code without human approval. Who is ethically responsible when that system causes harm (e.g., downtime, data loss, or biased behavior in production)? Should an AI ever have the authority to deploy to production? If so, under what safeguards or oversight? If not, why and where should the line be drawn between automation and accountability?
- If you could design the perfect AI-powered teammate to help your team with continuous integration and code quality, what would it do and what wouldn't it do? How would it interact with humans during reviews, testing, and deployments? Describe its features, personality, and limits. How would you ensure it improves collaboration instead of replacing it? Provide concrete examples.
- Monitoring cloud deployments can be difficult due to the number of replicated frontend and backend server functionality. Each one writes to its own log. Let's say you had an app that was scaled up to run on 100 servers. How would you use an LLM to make it easier and quicker to detect runtime problems from the logs as your app runs in the cloud? Provide three examples of different ways to analyze the logs to identify problems.

# Discussion Instructions

- Instructor announces the reflection question.
- If you chose this reflection question to answer for homework, come up to the front. You are the discussant group.
- Class forms into pairs.
- Pairs and discussant group should now reflect on the question by themselves (4 min)
  - Explain what you think, why, and how you've come to your conclusions.
- Each discussant explains their answer to the class (3-4 min)
- Instructor will choose a pair to report their answer to the class (1 min)
- A discussant will respond and connect the answer to their own reflection (1 min)

# Reflection Prompt

- Imagine your company uses an LLM-driven CI/CD system that can automatically decide when to deploy, roll back, or patch code without human approval. Who is ethically responsible when that system causes harm (e.g., downtime, data loss, or biased behavior in production)? Should an AI ever have the authority to deploy to production? If so, under what safeguards or oversight? If not, why and where should the line be drawn between automation and accountability?

# Reflection Prompt

Cloud providers like Amazon, Microsoft, and Google deliberately design their systems to promote vendor lock-in. This makes it very difficult to switch from one cloud to another. Speculate how you could use an LLM to support dynamically switching your app between cloud providers without bothering any human software developers. What would you need to do?

# Reflection Prompt

Could an LLM be integrated into your continuous integration pipeline to automatically review code, generate tests, or update tests whenever new code is committed? How would you do it? What are the potential benefits and risks to your production environment?

# Reflection Prompt

- Monitoring cloud deployments can be difficult due to the number of replicated frontend and backend server functionality. Each one writes to its own log. Let's say you had an app that was scaled up to run on 100 servers. How would you use an LLM to make it easier and quicker to detect runtime problems from the logs as your app runs in the cloud? Provide three examples of different ways to analyze the logs to identify problems.



# Reflection Prompt

- If you could design the perfect AI-powered teammate to help your team with continuous integration and code quality, what would it do and what wouldn't it do? How would it interact with humans during reviews, testing, and deployments? Describe its features, personality, and limits. How would you ensure it improves collaboration instead of replacing it? Provide concrete examples.

# Reflection Prompt

- Deployment to the cloud is much more complex than deploying on a single server in your garage. Ask your LLM to explain to you why this complexity is justified. After reading its explanation, engage your human brain to discuss two pros and two cons to cloud deployment vs. deployment on your own server.

# Next Class

- Monitoring Services
- Project Checkins